

UNIDAD 8. BASES DE DATOS NO RELACIONALES

-  UNIDAD 8. BASES DE DATOS NO RELACIONALES
 -  1. Introducción a las Bases de Datos No Relacionales
 -  2. Preparación del entorno de trabajo con MongoDB
 -  3. Comandos básicos desde la consola (MongoDB Shell)
 -  4. Operaciones básicas en MongoDB Compass
 -  5. Conociendo MongoDB y sus colecciones
 -  6. Insertar documentos
 -  7. Consultar documentos en MongoDB
 -  Consultas avanzadas
 -  Ejercicios (con solución)
 -  8. Actualizar documentos en MongoDB
 -  9. Eliminar documentos en MongoDB

1. Introducción a las Bases de Datos No Relacionales

¿Qué es una base de datos NoSQL?

Una base de datos NoSQL (Not Only SQL) es un sistema de almacenamiento de datos que **no sigue el modelo tradicional de tablas relacionadas** como en las bases de datos relacionales.

En lugar de trabajar con filas y columnas, estas bases de datos utilizan **estructuras más flexibles**, como documentos, clave-valor, grafos o columnas.

 La idea principal es sencilla: adaptarse mejor a aplicaciones modernas donde los datos pueden cambiar mucho o no tienen una estructura fija.

Diferencias básicas con bases de datos relacionales

Las bases de datos relacionales (como MySQL) organizan la información en tablas con una estructura fija. En cambio, las NoSQL son mucho más flexibles.

 Algunas diferencias clave:

- Las relacionales usan **tablas**, las NoSQL pueden usar **documentos (JSON)**
- En SQL necesitas definir la estructura antes, en NoSQL **puede cambiar fácilmente**
- Las relacionales usan relaciones (JOIN), en NoSQL se intenta **evitar ese tipo de operaciones**
- NoSQL está pensada para manejar **grandes volúmenes de datos** de forma eficiente

 En resumen: las bases NoSQL sacrifican algo de control estructural a cambio de **flexibilidad y rendimiento**.

Casos de uso

Las bases de datos NoSQL no sustituyen a las relacionales, sino que se utilizan cuando encajan mejor.

 Algunos ejemplos reales:

- Aplicaciones web con muchos usuarios (redes sociales, apps móviles)
- Sistemas con datos que cambian constantemente
- Aplicaciones que manejan grandes volúmenes de información (Big Data)
- APIs que trabajan con datos en formato JSON

Conclusión: trabajaremos con MongoDB

En este tema vamos a utilizar MongoDB, una de las bases de datos NoSQL más utilizadas.

 MongoDB almacena la información en **documentos tipo JSON**, lo que permite trabajar de forma muy parecida a como lo hacemos en muchos lenguajes de programación.

 En los siguientes apartados aprenderás paso a paso:

- Cómo instalar MongoDB
- Cómo crear bases de datos
- Cómo insertar, consultar y modificar datos

RECUERDA

- NoSQL ≠ "sin SQL", significa "no solo SQL"
- Mayor flexibilidad en los datos
- Muy útil en aplicaciones modernas

2. Preparación del entorno de trabajo con MongoDB

Antes de empezar a trabajar con datos, necesitamos preparar nuestro entorno. Vamos a hacerlo paso a paso.

Instalación de MongoDB

MongoDB se puede instalar de forma local en tu equipo.

 Pasos generales:

1. Accede a la web oficial de MongoDB
2. Descarga la versión Community Server
3. Ejecuta el instalador
4. Sigue la instalación con las opciones por defecto

 Durante la instalación:

- Se instalará también el servicio de MongoDB
- En algunos casos se añadirá automáticamente al sistema

 Consejo: no cambies rutas ni configuraciones si no tienes experiencia previa.

Compass / Shell

MongoDB se puede usar de dos formas principales:

♦ **MongoDB Compass** (interfaz gráfica)

Es una aplicación visual que permite trabajar con bases de datos sin escribir comandos.

👉 Ideal para empezar:

- Ver datos fácilmente
- Crear colecciones
- Insertar documentos con formularios

♦ MongoDB Shell (mongosh)

Es la consola de comandos de MongoDB.

👉 Permite:

- Ejecutar consultas
- Automatizar tareas
- Tener un mayor control

🧠 Lo habitual es combinar ambos:

usar Compass para visualizar y el Shell para practicar comandos.

▶ Puesta en marcha del servidor

MongoDB funciona como un servicio que debe estar en ejecución.

👉 En la mayoría de instalaciones:

- En **Windows**, se inicia automáticamente como servicio
- En **Linux/Mac**, puede ser necesario iniciarlo manualmente

Ejemplo en Linux/Mac:

```
sudo systemctl start mongod
```

Para comprobar que está funcionando:

```
sudo systemctl status mongod
```

🔧 Primera conexión

Una vez el servidor está en marcha, podemos conectarnos.

✔ Con MongoDB Compass

Abre MongoDB Compass

Usa la conexión por defecto:

mongodb://localhost:27017

Pulsa en Connect

👉 Si todo va bien, verás las bases de datos disponibles.

✓ Con MongoDB Shell

Abre una terminal y escribe:

```
mongosh
```

✚ Si la conexión es correcta, verás algo como:

```
test>
```

Eso indica que ya estás dentro de MongoDB.

🧪 Comprueba que todo funciona

Una vez conectado, prueba este comando:

```
show dbs
```

👉 Si aparece una lista (aunque esté vacía), todo está funcionando correctamente.

📄 RECUERDA

- MongoDB necesita un servidor en ejecución
- Puedes trabajar con interfaz gráfica (Compass) o consola (mongosh)
- La conexión local usa: `mongodb://localhost:27017`

💡 Nuestro escenario: instalamos MongoDB en una máquina virtual en Windows 10 o Windows 11 y usamos el cliente desde la máquina real.

Para la instalación, ponemos la máquina virtual en Adaptador-puente, descargar el fichero correspondiente y lo instalas.

Después de la descarga, vuelves a modo solo-anfitrión e intentas conectarte desde la máquina real, pero antes hay que hacer lo siguiente:

1. 📄 Modificar el archivo de configuración de MongoDB

En la máquina virtual (Windows 10), busca el archivo:

`C:\Program Files\MongoDB\Server<versión>\bin\mongod.cfg`

Busca esta parte:

`net:`

`bindIp: 127.0.0.1`

👉 Cámbiala por:

net:

bindIp: 0.0.0.0

✦ Esto permite que MongoDB escuche en todas las interfaces de red.

2.  Reiniciar el servicio de MongoDB

En Windows:

Abre services.msc

Busca MongoDB Server

Pulsa en Reiniciar

3.  Comprobar conectividad desde la máquina real

Desde tu máquina física prueba:

ping 192.168.56.106

4.  Conexión correcta en MongoDB

Te conectas desde Mongo Compass desde tu máquina real.



3. Comandos básicos desde la consola (MongoDB Shell)

Una vez dentro de MongoDB usando **mongosh**, podemos empezar a trabajar con comandos básicos para gestionar bases de datos y consultar información.

Para entrar en la consola:

mongosh



Gestión de bases de datos

Para ver todas las bases de datos disponibles en el servidor:

```
show dbs
```

Este comando muestra únicamente las bases de datos que contienen datos.

Para ver en qué base de datos estamos trabajando en este momento:

```
db
```

Para cambiar de base de datos o crear una nueva:

```
use nombreBaseDatos
```

✦ Si la base de datos no existe, se creará automáticamente cuando insertes el primer dato.

Ejemplo:

```
use tienda
```

📁 Gestión de colecciones

Las colecciones son el equivalente a las tablas en bases de datos relacionales.

Para ver las colecciones de la base de datos actual:

```
show collections
```

📄 Consulta de datos

Para consultar todos los documentos de una colección:

```
db.nombreColeccion.find()
```

Ejemplo:

```
db.productos.find()
```

Para ver los resultados con un formato más legible:

```
db.nombreColeccion.find().pretty()
```

Para limitar el número de resultados:

```
db.nombreColeccion.find().limit(5)
```

🧰 4. Operaciones básicas en MongoDB Compass

MongoDB Compass es la herramienta gráfica de MongoDB que permite trabajar con bases de datos sin necesidad de usar comandos. Es muy útil para visualizar datos, hacer consultas y gestionar colecciones de forma intuitiva.

🔌 Conexión al servidor

Al abrir Compass, lo primero es conectarse al servidor.

En el campo de conexión se introduce:

mongodb://localhost:27017

Después se pulsa en **Connect** para acceder al servidor.

Si la conexión es correcta, aparecerán las bases de datos disponibles en el panel lateral.

Visualizar bases de datos

En la interfaz principal, en la parte izquierda, se muestran todas las bases de datos existentes.

Desde ahí puedes:

- Ver las bases de datos disponibles
- Entrar en una base de datos haciendo clic sobre su nombre
- Ver el tamaño y número de colecciones

Ver colecciones

Dentro de cada base de datos aparecen sus colecciones.

Al hacer clic en una colección puedes:

- Ver los documentos almacenados
- Navegar por los registros
- Ver la estructura de los datos (formato JSON)

Consultar datos

Dentro de una colección puedes ver los documentos directamente en pantalla.

Además, Compass permite filtrar datos usando la barra de **Filter**.

Ejemplos de filtros:

- Buscar todos los documentos:
`{}`
- Buscar por campo:
`{"nombre": "Juan" }`
- Buscar por condiciones:
`{"edad": { "$gt": 18 } }`

También puedes:

- Ordenar resultados
- Limitar la visualización de campos
- Aplicar filtros avanzados sin escribir código complejo

Insertar documentos

Compass permite insertar datos de forma visual:

1. Entra en una colección
2. Pulsa en **Add Data**
3. Selecciona **Insert Document**
4. Escribe el documento en formato JSON
5. Guarda los cambios

Ejemplo de documento:

```
{  
  "nombre": "Ana",  
  "edad": 25,  
  "ciudad": "Lugo"  
}
```

Editar y eliminar datos

Desde cada documento puedes:

- Editar campos directamente
- Guardar cambios con un clic
- Eliminar documentos individuales

Esto facilita mucho el trabajo sin necesidad de comandos.

HOJAS DE EJERCICIOS

 Hoja de ejercicios 1.

5. Conociendo MongoDB y sus colecciones

MongoDB organiza la información de una forma diferente a las bases de datos relacionales. En lugar de tablas y filas, trabaja con una estructura más flexible basada en **bases de datos, colecciones y documentos**.

Una **base de datos en MongoDB** es simplemente un contenedor donde se guarda la información. Dentro de ella no hay tablas, sino colecciones. Puedes tener tantas bases de datos como necesites, cada una destinada a un propósito distinto, como por ejemplo una para una tienda, otra para una biblioteca o una aplicación web.

Dentro de cada base de datos encontramos las **colecciones**, que son el equivalente a las tablas en los sistemas relacionales. Sin embargo, aquí no existe una estructura rígida: una colección puede almacenar documentos diferentes entre sí, lo que permite una mayor flexibilidad. Por ejemplo, en una colección de productos, algunos documentos pueden tener más campos que otros.

Los **documentos** son la unidad básica de información en MongoDB. Se almacenan en formato similar a JSON, lo que hace que sean fáciles de entender y de usar desde lenguajes de programación. Cada documento representa un registro y está formado por pares **clave-valor**.

 Por ejemplo, un documento podría ser:


```
{
  "nombre": "Portátil",
  "precio": 800,
  "stock": 10
}
```

En cuanto a la **navegación básica**, trabajar con MongoDB implica moverse entre bases de datos y colecciones. Primero se selecciona una base de datos, después una colección, y finalmente se consultan o modifican los documentos que contiene. Este flujo es sencillo y muy directo, tanto desde consola como desde herramientas gráficas.

Ejemplo completo

Imagina una base de datos llamada **tienda**:

-  Colección: productos
-  Documento:

```
{
  "nombre": "Ratón",
  "precio": 25,
  "categoria": "informatica"
}
```

 En este caso:

- **tienda** es la base de datos
- **productos** es la colección
- El JSON es el documento

6. Insertar documentos

Una de las operaciones más importantes en MongoDB es la inserción de datos. Como ya sabes, la información se guarda en **documentos**, y estos se almacenan dentro de colecciones. Para añadir nuevos datos utilizamos comandos sencillos desde la consola (**mongosh**) o herramientas gráficas como MongoDB Compass.

El método más básico es **insertOne()**, que permite añadir un único documento a una colección. Es útil cuando queremos introducir datos de forma puntual o probar ejemplos pequeños.

 Por ejemplo:

```
db.productos.insertOne({
  nombre: "Portátil",
  precio: 800,
  stock: 10
})
```

Si necesitamos insertar varios documentos a la vez, utilizamos **insertMany()**. Este método recibe un conjunto de documentos y los guarda todos en una sola operación, lo que resulta más eficiente cuando trabajamos con varios registros.

👉 Ejemplo:

```
db.productos.insertMany([
  { nombre: "Ratón", precio: 25, stock: 50 },
  { nombre: "Teclado", precio: 60, stock: 30 }
])
```

En cuanto a los **tipos de datos**, MongoDB trabaja con valores similares a JSON. Los más habituales son **cadenas de texto**, **números**, **booleanos** y también estructuras más complejas como listas o documentos anidados. Esto permite representar información de forma muy natural.

👉 Ejemplo con distintos tipos:

```
{
  nombre: "Monitor",
  precio: 200,
  disponible: true,
  etiquetas: ["pantalla", "oficina"]
}
```

🧪 Ejercicios guiados

A continuación, practica paso a paso en la **consola**:

- Crea una base de datos llamada **empresa** y una colección llamada **empleados**
- Inserta un empleado con nombre, edad y puesto
- Inserta varios empleados con diferentes datos
- Añade un campo nuevo en alguno de los documentos (por ejemplo, departamento)

📄 RECUERDA

- `insertOne()` → inserta un documento
- `insertMany()` → inserta varios documentos
- Los datos se guardan en formato tipo JSON
- MongoDB permite estructuras flexibles y dinámicas

HOJAS DE EJERCICIOS

📄 Hoja de ejercicios 2.

🔍 7. Consultar documentos en MongoDB

Consultar información en MongoDB es una de las operaciones más importantes, ya que permite recuperar y filtrar datos almacenados en las colecciones de forma muy flexible. El método principal es **find()**, pero a partir de él se pueden construir consultas cada vez más potentes utilizando operadores.

🔍 `find()` — base de todas las consultas

El método **find()** permite buscar documentos dentro de una colección. Si no se indica ninguna condición, devuelve todos los registros.

👉 Ejemplo:

```
db.productos.find()
```

Este comando muestra todos los documentos almacenados en la colección.

🎯 Búsquedas simples

Las búsquedas simples permiten filtrar documentos indicando valores concretos de los campos.

👉 Ejemplo:

```
db.productos.find({ categoria: "informatica" })
```

También podemos filtrar por valores numéricos:

```
db.productos.find({ precio: 80 })
```

🎭 Proyecciones

Las proyecciones permiten decidir qué campos se muestran en el resultado de la consulta.

👉 Ejemplo:

```
db.productos.find({}, { nombre: 1, precio: 1, _id: 0 })
```

Esto muestra únicamente el nombre y el precio de los productos.

🧠 Consultas avanzadas

MongoDB permite realizar consultas más potentes mediante operadores que amplían mucho las posibilidades de filtrado.

⚖️ Operadores de comparación

Los operadores permiten comparar valores:

- \$gt → mayor que
- \$lt → menor que
- \$gte → mayor o igual que
- \$lte → menor o igual que
- \$ne → distinto de

👉 Ejemplo:

```
db.productos.find({ precio: { $gt: 100 } })
```

🔗 Condiciones múltiples (AND implícito)

Cuando se incluyen varios campos en la misma consulta, MongoDB interpreta una condición AND.

👉 Ejemplo:

```
db.productos.find({ categoria: "informatica", precio: { $lt: 100 } })
```

🔗 Operador OR

El operador **\$or** permite cumplir una condición u otra.

👉 Ejemplo:

```
db.productos.find({
  $or: [
    { categoria: "informatica" },
    { precio: { $lt: 50 } }
  ]
})
```

Esto devuelve productos de informática o productos con precio menor de 50.

🚫 Operador NOT

El operador **\$not** se utiliza para negar una condición.

👉 Ejemplo:

```
db.productos.find({
  precio: { $not: { $gt: 100 } }
})
```

Esto devuelve productos cuyo precio **no sea mayor de 100**.

📦 Operador IN

El operador **\$in** permite buscar valores dentro de una lista.

👉 Ejemplo:

```
db.productos.find({
  categoria: { $in: ["informatica", "muebles"] }
})
```

Esto devuelve productos cuya categoría sea informática o muebles.

🧪 Ejercicios (con solución)

● Muestra todos los productos de la colección

✓ Solución: `db.productos.find()`

● Muestra los productos de la categoría "informatica"

✓ Solución: `db.productos.find({ categoria: "informatica" })`

● Muestra los productos con precio mayor de 50

✓ Solución: `db.productos.find({ precio: { $gt: 50 } })`

- Muestra los productos con precio entre 50 y 150
- ✓ Solución: `db.productos.find({ precio: { $gte: 50, $lte: 150 } })`
- Muestra los productos de informática o muebles
- ✓ Solución: `db.productos.find({ categoria: { $in: ["informatica", "muebles"] } })`
- Muestra productos que NO sean de la categoría informática
- ✓ Solución: `db.productos.find({ categoria: { $ne: "informatica" } })`
- Muestra productos de informática o con precio menor de 50
- ✓ Solución:
`db.productos.find({
 $or: [
 { categoria: "informatica" },
 { precio: { $lt: 50 } }
]
})`

HOJAS DE EJERCICIOS

 Hoja de ejercicios 3.

8. Actualizar documentos en MongoDB

Actualizar documentos en MongoDB permite **modificar datos ya existentes** sin necesidad de eliminarlos. Para ello se utilizan principalmente los métodos **updateOne()** y **updateMany()**, junto con operadores específicos como **\$set**, **\$unset** y **\$inc**.

updateOne()

El método **updateOne()** modifica **un único documento** que cumpla una condición.

👉 Ejemplo:

```
db.productos.updateOne(  
  { nombre: "Portátil HP" },  
  { $set: { precio: 700 } }  
)
```

- ✓ Solo actualiza el primer documento que coincide con la condición.

updateMany()

El método **updateMany()** permite modificar **varios documentos a la vez**.

👉 Ejemplo:

```
db.productos.updateMany(  
  { categoria: "informatica" },  
  { $set: { stock: 20 } }  
)
```

- ✓ Afecta a todos los documentos que cumplen la condición.

🧩 Operadores de actualización

🟢 \$set → **modificar o añadir campos**

```
db.productos.updateOne(
  { nombre: "Silla gaming" },
  { $set: { stock: 10 } }
)
```

- ✓ Cambia el valor del campo o lo crea si no existe.

🔴 \$unset → **eliminar campos**

```
db.productos.updateOne(
  { nombre: "Auriculares" },
  { $unset: { stock: "" } }
)
```

- ✓ Elimina un campo del documento.

+ \$inc → **incrementar valores**

```
db.productos.updateOne(
  { nombre: "Teclado mecánico" },
  { $inc: { stock: 5 } }
)
```

- ✓ Suma o resta valores numéricos.

🧪 Ejercicios prácticos (con solución)

🟢 Aumenta el precio de todos los productos en 10 euros

- ✓ Solución:

```
db.productos.updateMany({}, { $inc: { precio: 10 } })
```

🟢 Cambia la categoría del producto "Silla gaming" a "tecnologia"

- ✓ Solución:

```
db.productos.updateOne(
  { nombre: "Silla gaming" },
  { $set: { categoria: "tecnologia" } }
)
```

🟢 Añade un campo "oferta: true" a todos los productos con precio menor de 100

- ✓ Solución:

```
db.productos.updateMany(
  { precio: { $lt: 100 } },
  { $set: { oferta: true } }
)
```

● Elimina el campo "stock" del producto "Auriculares"

✓ Solución:

```
db.productos.updateOne(  
  { nombre: "Auriculares" },  
  { $unset: { stock: "" } }  
)
```



9. Eliminar documentos en MongoDB

Eliminar documentos permite **borrar registros de una colección**. Es una operación potente y debe usarse con cuidado.



`deleteOne()`

El método **deleteOne()** elimina **un único documento** que cumpla una condición.

👉 Ejemplo:

```
db.productos.deleteOne({ nombre: "Ratón inalámbrico" })
```

✓ Solo elimina el primer documento que coincide.



`deleteMany()`

El método **deleteMany()** elimina **todos los documentos que cumplan una condición**.

👉 Ejemplo:

```
db.productos.deleteMany({ categoria: "muebles" })
```

✓ Elimina todos los documentos de esa categoría.



Buenas prácticas

- 🔍 Comprobar siempre con `find()` antes de borrar
- 🎯 Usar condiciones precisas para evitar borrados masivos
- 🧪 Probar primero con pocos datos
- 💾 Evitar eliminaciones sin copia de seguridad



Ejercicios (con solución)

● Elimina el producto "Teclado mecánico"

✓ Solución:

```
db.productos.deleteOne({ nombre: "Teclado mecánico" })
```

● Elimina todos los productos con stock igual a 0

✓ Solución:

```
db.productos.deleteMany({ stock: 0 })
```

● Elimina todos los productos de la categoría "muebles"

✓ Solución:

```
db.productos.deleteMany({ categoria: "muebles" })
```

● Comprueba primero con `find()` y luego elimina los productos con precio mayor de 500

✓ Solución (consulta previa):

```
db.productos.find({ precio: { $gt: 500 } })
```

✓ Solución (borrado):

```
db.productos.deleteMany({ precio: { $gt: 500 } })
```

🧠 RECUERDA

- **updateOne()** → modifica un documento
- **updateMany()** → modifica varios documentos
- **\$set, \$unset, \$inc** → operadores de actualización
- **deleteOne()** → elimina un documento
- **deleteMany()** → elimina varios documentos
- Siempre comprobar antes de borrar datos

HOJAS DE EJERCICIOS

📄 Hoja de ejercicios 4.

📄 Hoja de ejercicios 5.